Reporte

Nombre: Roberto Carlos Sánchez Flores Donovan Adrian Serrato Lores Laura Sofía Vargas Ramírez	**Matrícula**: 2151262 2154304 2152053
Nombre del curso: Componentes de lógica de negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 5	**Actividad**: Laboratorio guiado · Unidad 5: Integración, pruebas y calidad
Fecha: 05/11/2026	
Bibliografia (Formato APA): Documentos dados por el docente: Manual autónomo del alumno U5 (versión corregida), Manual del alumno U5, README de la Unidad 5, presentación ampliada U5, paquete CLN_U5_Archivos_Completos_Laboratorio y paquete de datos del PIA de CleanPro. Beck, K. (2002). Test-driven development: By example. Addison-Wesley. Meszaros, G. (2007). xUnit test patterns: Refactoring test code. Addison-Wesley. pytest-dev. (2026). pytest documentation. https://docs.pytest.org/ SQLAlchemy. (2026). SQLAlchemy 2.1 documentation. https://docs.sqlalchemy.org/ Batchelder, N. (2026). Coverage.py documentation. https://coverage.readthedocs.io/ GitHub. (2026). GitHub Actions documentation. https://docs.github.com/actions	

Objetivo:

Comprobar con pruebas automatizadas que las reglas de negocio de un sistema de pedidos se cumplen, que la persistencia y los servicios externos se comportan como se espera y que GitHub rechaza los cambios que rompen las pruebas. A partir del proyecto base de la Unidad 5 se distinguen pruebas unitarias, de integración y extremo a extremo; se aplica Arrange–Act–Assert y el ciclo TDD Red–Green–Refactor; se verifican commit y rollback con SQLAlchemy sin tocar datos de producción; se simulan cobros con mocks; se prueba la autorización; se mide la cobertura (≥80 %) y se automatiza la revisión con GitHub Actions. Finalmente, lo aprendido se transfiere a tres reglas del BRD de CleanPro, la empresa del PIA del equipo.

Procedimiento:

Integración, pruebas y calidad del sistema de pedidos

Una empresa recibe pedidos: un pedido VIP recibe 15 % de descuento; no puede confirmarse si su importe es cero o negativo o si el cliente tiene deuda; el cobro se hace con una pasarela de pago; los pedidos se guardan en una base de datos y hay reglas para cancelarlos. Una falla en cualquiera de esas reglas puede generar pérdidas, cobros incorrectos o accesos indebidos. El trabajo no consistió en construir una aplicación nueva, sino en demostrar con pruebas que la existente funciona, y en mejorarla con TDD donde una prueba descubrió un defecto.

Estrategia de pruebas

Tipo	Qué verifica	Dependencias	En este proyecto
Unitaria	Una regla del dominio de forma aislada	Ninguna: sin base de datos ni red	test_order, test_security, test_casos_csv y las reglas de CleanPro
Integración	Que el código funciona junto con una pieza real de infraestructura	SQLite en memoria vía SQLAlchemy	test_repository y test_cleanpro_integracion (commit, rollback, unicidad)
Con dobles (mock)	Cómo reacciona el servicio ante un sistema externo	Mock que sustituye la pasarela o el notificador	test_service y el notificador de CleanPro
Extremo a extremo (E2E)	Un flujo completo desde la interfaz del usuario hasta la base de datos	API o interfaz, base de datos y servicios	No aplica: el proyecto base no tiene API ni interfaz (ver abajo)

Tabla 1. *Tipos de prueba y su uso en el proyecto.*

La pirámide de pruebas (70 % unitarias, 20 % integración, 10 % E2E) se usó como referencia: la mayoría de las pruebas son de dominio porque son rápidas y señalan con precisión qué regla falla. **Escenario E2E propuesto** (documentado, no implementado): en el sistema de CleanPro del PIA, un cliente solicita un servicio por la API REST, el coordinador lo programa, el supervisor lo completa y administración genera la factura; la prueba recorrería los endpoints con un cliente HTTP y verificaría en la base de datos el estado final y el monto facturado.

Paso 1 · Instalar y comprobar el proyecto (E1)

Se descomprimió el proyecto base en una carpeta independiente, se creó el entorno virtual y se instalaron las dependencias (pytest, pytest-cov, SQLAlchemy y Ruff). Se usó Python 3.13.17, compatible con el requisito de Python 3.11 o superior.

Terminal (PowerShell)

```
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python -m pytest -v
```

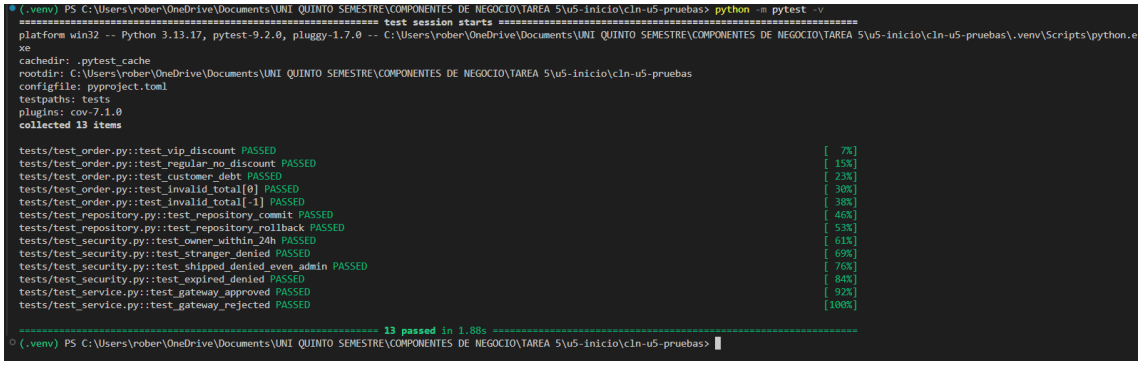


Figura 1. *Ejecución inicial de las 13 pruebas del proyecto base: todas en PASSED.*

Evidencia E1.

pytest localiza automáticamente las funciones cuyo nombre empieza con test_, las ejecuta y resume cuáles pasaron o fallaron, lo que convierte la revisión manual en una verificación repetible. El entorno virtual (.venv) aísla las bibliotecas y versiones del proyecto de las del resto de la computadora. Así cualquier integrante, o GitHub Actions, puede reproducir exactamente el mismo resultado.

Antes de continuar se revisó el proyecto base y se hicieron tres ajustes mínimos de configuración que **no cambian ninguna regla de negocio**:

Hallazgo	Causa	Ajuste
Con pytest a secas (como lo ejecuta GitHub Actions) aparecía ModuleNotFoundError: No module named 'src'	Las pruebas importan src.domain, pero pytest solo agrega la carpeta tests al path; con python -m pytest funcionaba porque Python agrega la carpeta actual	pythonpath = ["."] en pyproject.toml
Ruff marcaba 7 errores I001 (imports desordenados) en el código original	Las versiones recientes de Ruff activan más reglas por defecto	ruff check --fix y reglas fijas E, F, I en pyproject.toml, para que el resultado no cambie con cada versión
Aviso ResourceWarning: unclosed database al terminar	Las dos pruebas originales del repositorio no cerraban el motor de la base en memoria	engine.dispose() al final de cada una; las nuevas usan un fixture

Tabla 2. *Ajustes al proyecto base antes de empezar.*

Paso 2 · Comprender una prueba unitaria (E2)

Order.confirm() rechaza totales ≤ 0, rechaza clientes con deuda, calcula el descuento VIP del 15 % y cambia el estado a CONFIRMED. Las dos pruebas nuevas siguen el patrón Arrange–Act–Assert: se prepara el pedido, se ejecuta la confirmación y se verifican el cobro, el descuento y el estado. No requieren base de datos ni servicios externos porque solo evalúan la regla.

tests/test_order.py

```
def test_vip_200_paga_170():
    order = Order(200, vip=True)  # Arrange: pedido VIP de $200
    amount = order.confirm()      # Act: confirmar
    assert amount == 170          # Assert: 15% de descuento
    assert order.discount == 30
    assert order.status == "CONFIRMED"


def test_regular_200_paga_200():
    order = Order(200)            # Arrange: cliente regular
    amount = order.confirm()      # Act
    assert amount == 200          # Assert: sin descuento
    assert order.discount == 0
```

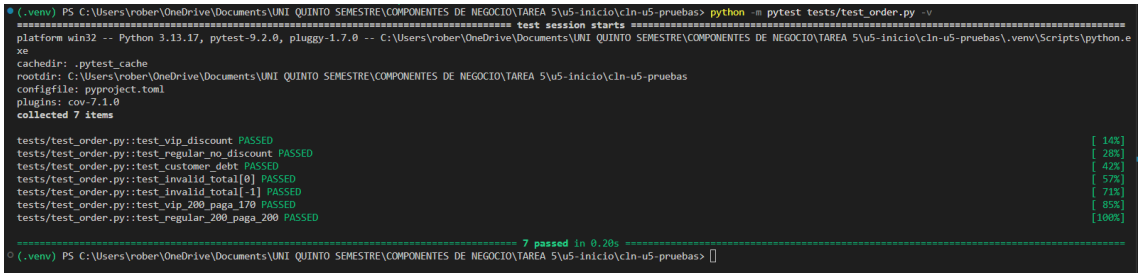


Figura 2. *Ejecución de tests/test_order.py con las dos pruebas nuevas: 7 passed.*

Evidencia E2.

Prueba	Entrada	Regla	Esperado	Real
test_vip_200_paga_170	total = 200, VIP	VIP recibe 15 % de descuento	Cobro 170, descuento 30, CONFIRMED	170 · 30 · CONFIRMED (PASSED)
test_regular_200_paga_200	total = 200, regular	Sin VIP no hay descuento	Cobro 200, descuento 0	200 · 0 (PASSED)

Tabla 3. *Entrada, regla, resultado esperado y resultado real de las pruebas nuevas.*

Paso 3 · TDD real: Red → Green → Refactor (E3)

La regla elegida fue **no confirmar un pedido que ya fue confirmado**. El proyecto ya validaba el total positivo, así que repetir esa regla no habría producido la fase roja. El caso CP08 del paquete (un pedido que ya está en CONFIRMED) mostraba el comportamiento original: se podía confirmar dos veces.

RED. Primero se escribió la prueba, sin tocar src/domain.py:

tests/test_order.py

```
def test_cannot_confirm_twice():
    order = Order(100)
    order.confirm()
    with pytest.raises(ValueError, match="already confirmed"):
        order.confirm()
```

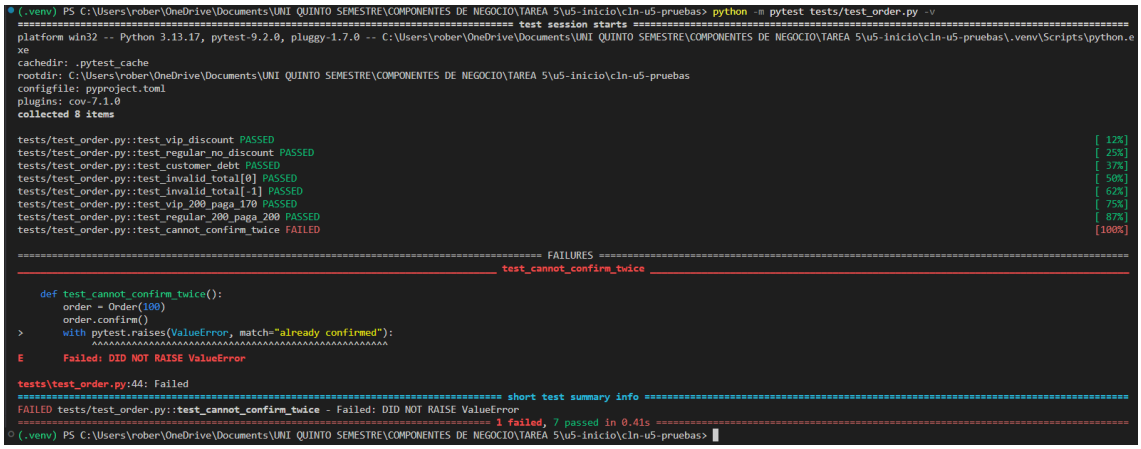


Figura 3. *Fase RED: la prueba nueva falla con DID NOT RAISE ValueError (1 failed, 7 passed).*

GREEN. Se agregó la implementación mínima al inicio de confirm():

src/domain.py — fase GREEN

```
def confirm(self, customer_debt=0):
    if self.status == "CONFIRMED":
        raise ValueError("already confirmed")
    ...
```

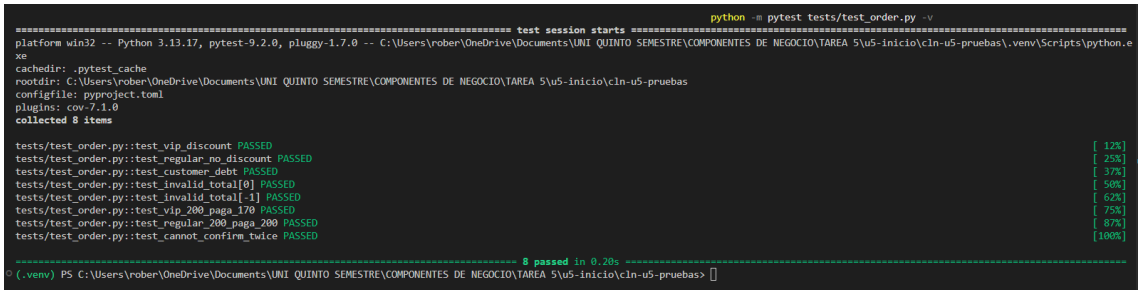


Figura 4. *Fase GREEN: la prueba pasa (8 passed).*

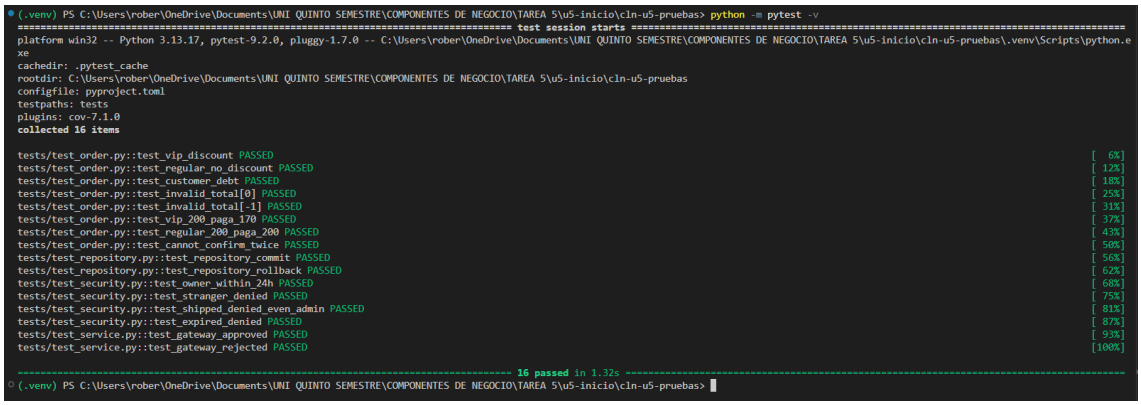


Figura 5. *Suite completa después de GREEN: las 16 pruebas siguen pasando.*

REFACTOR. Se extrajeron las validaciones a un método privado y se dio un nombre claro a la excepción, manteniendo la interfaz pública: OrderAlreadyConfirmed es subclase de ValueError y conserva el mensaje «already confirmed», por lo que la misma prueba sigue pasando sin modificarse.

src/domain.py — fase REFACTOR

```
class OrderAlreadyConfirmed(ValueError):
    """Regla agregada con TDD: un pedido no puede confirmarse dos veces."""


def confirm(self, customer_debt=0):
    self._ensure_can_be_confirmed(customer_debt)
    self.discount = round(self.total * VIP_DISCOUNT_RATE, 2) if self.vip else 0.0
    self.status = "CONFIRMED"
    return self.total - self.discount


def _ensure_can_be_confirmed(self, customer_debt):
    if self.status == "CONFIRMED":
        raise OrderAlreadyConfirmed("already confirmed")
    if self.total <= 0:
        raise ValueError("total must be positive")
    if customer_debt > 0:
        raise CustomerHasDebt("outstanding debt")
```

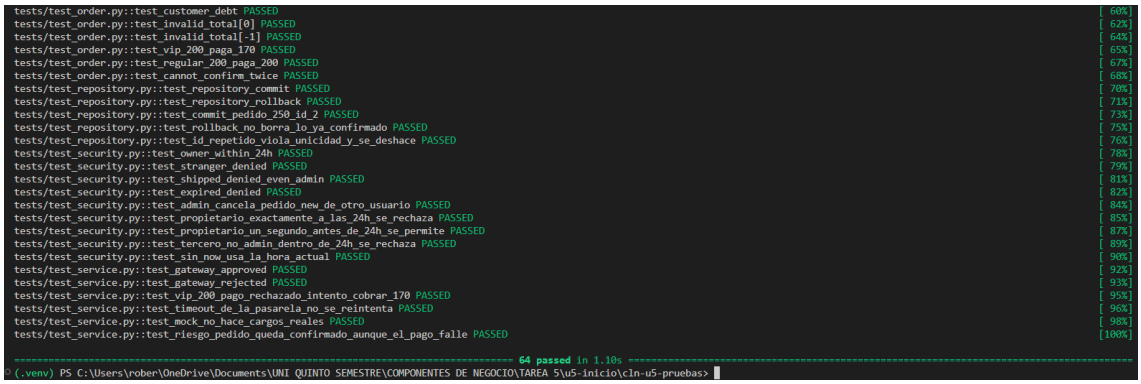


Figura 6. *Después del refactor y de agregar el resto de las pruebas: 64 passed.*

Evidencia E3.

TDD no es ejecutar pruebas después de programar. La prueba se escribe antes y funciona como especificación: primero demuestra que el defecto existe (fase roja), luego guía el cambio mínimo que lo corrige (verde) y finalmente protege el comportamiento mientras se mejora el diseño (refactor). Si la prueba se escribe después, es fácil acomodarla a lo que el código ya hace y nunca verla fallar. En ese caso no hay evidencia de que la prueba detecte el defecto.

Paso 4 · Integración con base de datos y transacciones (E4)

SqlOrderRepository.save() agrega el pedido a la sesión de SQLAlchemy; session.commit() lo hace permanente en la base de prueba y session.rollback() lo descarta. Cada prueba crea su propia base SQLite **en memoria** mediante un fixture que la destruye al terminar, de modo que nunca se tocan datos de producción ni datos de otra prueba.

tests/conftest.py y tests/test_repository.py

```
@pytest.fixture
def session():
    engine = make_engine()              # SQLite en memoria
    with Session(engine) as s:
        yield s
    engine.dispose()


def test_commit_pedido_250_id_2(session):
    SqlOrderRepository(session).save(2, Order(250))
    session.commit()
    row = session.scalar(select(OrderRow).where(OrderRow.id == 2))
    assert (row.id, row.total, row.status) == (2, 250, "NEW")


def test_id_repetido_viola_unicidad_y_se_deshace(session):
    repo = SqlOrderRepository(session)
    repo.save(1, Order(120)); session.commit()
    repo.save(1, Order(999))
    with pytest.raises(IntegrityError):
        session.commit()
    session.rollback()
    assert session.scalar(select(OrderRow).where(OrderRow.id == 1)).total == 120
```

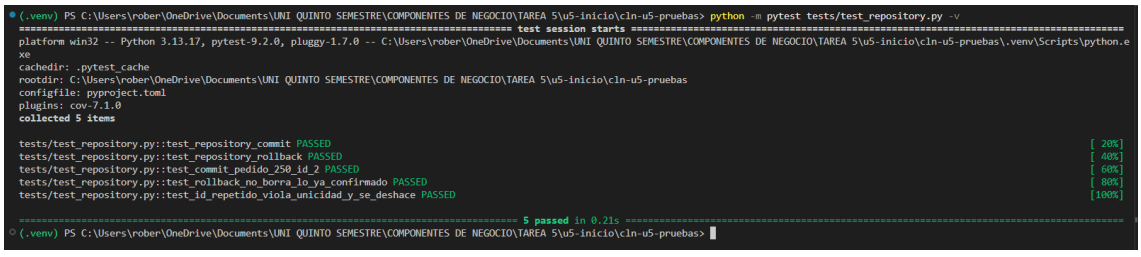


Figura 7. *Pruebas de persistencia: 5 passed.*

Evidencia E4.

Un **commit** confirma la transacción: los cambios quedan guardados y visibles para consultas posteriores. Un **rollback** la deshace: todo lo agregado desde el último commit se descarta, pero lo confirmado antes se conserva

(test_rollback_no_borra_lo_ya_confirmado). La prueba de unicidad muestra el uso más importante del rollback: cuando la base rechaza un id repetido, la transacción se revierte y el pedido original queda intacto. SQLite en memoria es rápido y aislado, pero no sustituye todas las pruebas contra un motor productivo como PostgreSQL.

Paso 5 · Servicios externos sin cobros reales (E5)

OrderService.checkout() confirma el pedido y llama a la pasarela de pago. En las pruebas, la pasarela se sustituye por un Mock que responde lo que la prueba decide; no se transmite dinero ni se llama a ningún proveedor.

tests/test_service.py

```
def test_vip_200_pago_rechazado_intento_cobrar_170():
    gateway = Mock()
    gateway.charge.return_value = False
    with pytest.raises(RuntimeError, match="payment rejected"):
        OrderService(gateway).checkout(Order(200, vip=True))
    gateway.charge.assert_called_once_with(170)


def test_timeout_de_la_pasarela_no_se_reintenta():
    gateway = Mock()
    gateway.charge.side_effect = TimeoutError("gateway timeout")
    with pytest.raises(TimeoutError):
        OrderService(gateway).checkout(Order(100))
    assert gateway.charge.call_count == 1


def test_riesgo_pedido_queda_confirmado_aunque_el_pago_falle():
    gateway = Mock(); gateway.charge.return_value = False
    order = Order(100)
    with pytest.raises(RuntimeError):
        OrderService(gateway).checkout(order)
    assert order.status == "CONFIRMED"   # riesgo documentado
```

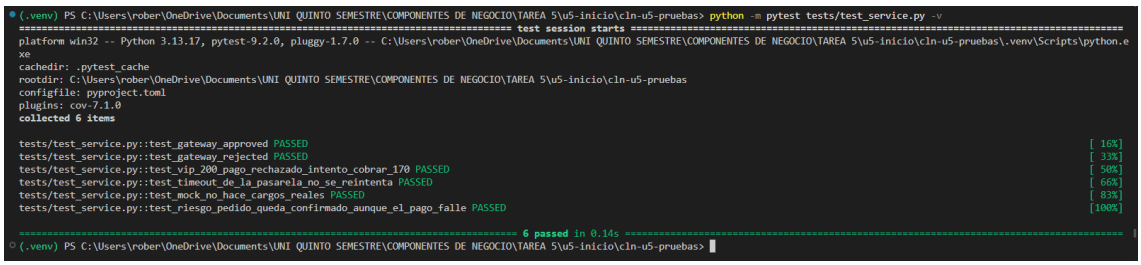


Figura 8. *Pruebas con la pasarela simulada: 6 passed.*

Evidencia E5.

Ruta	Mock	Llamada esperada	Resultado
Pago aprobado	charge → True	charge(100) una vez	checkout devuelve 100
Pago rechazado (VIP 200)	charge → False	charge(170) una vez	RuntimeError: payment rejected
Pasarela sin respuesta	charge → TimeoutError	charge una vez, sin reintentos	TimeoutError se propaga

Ruta	Mock	Llamada esperada	Resultado
Sin cargos reales	charge → True	method_calls == [call.charge(100)]	Única interacción: la simulada

Tabla 4. *Rutas probadas con el doble de la pasarela de pago.*

Riesgo de consistencia encontrado. El código base confirma el pedido antes de saber si el pago fue aceptado; la prueba de caracterización lo demuestra: después del rechazo el pedido queda en CONFIRMED. **Propuesta de mejora:** separar el cálculo del monto de la confirmación. El servicio calcularía primero el importe sin cambiar el estado, cobraría y solo si la pasarela aprueba confirmaría el pedido, todo dentro de una misma unidad de trabajo. Si el cobro falla o se agota el tiempo, la transacción se revierte. Si el pedido ya se guardó, se puede usar un estado intermedio PENDIENTE_DE_PAGO con una compensación que regrese el pedido a NEW. No se cambió el código base para mantener el alcance del laboratorio, y el riesgo queda documentado por la prueba.

Paso 6 · Seguridad y autorización (E6)

can_cancel aplica las reglas en este orden: un pedido SHIPPED nunca se cancela, ni siquiera por un administrador; en los demás estados el administrador puede cancelar; un propietario que no es administrador solo puede cancelar durante las primeras 24 horas. Las pruebas nuevas usan una fecha fija y pasan now explícitamente, para que el resultado no dependa del reloj.

tests/test_security.py

```
AHORA = datetime(2026, 11, 3, 12, 0, tzinfo=timezone.utc)

def test_admin_cancela_pedido_new_de_otro_usuario():
    assert can_cancel("NEW", 1, 2, True, AHORA - timedelta(hours=48), AHORA)

def test_propietario_exactamente_a_las_24h_se_rechaza():
    assert not can_cancel("NEW", 1, 1, False, AHORA - timedelta(hours=24), AHORA)
```

Rol	Dueño	Pedido	Antigüedad	Esperado	Real	Prueba
Propietario	Sí	NEW	1 h	Permitido	Permitido	PASA
Tercero	No	NEW	0 h	Denegado	Denegado	PASA
Administrador	Sí	SHIPPED	0 h	Denegado	Denegado	PASA
Propietario	Sí	NEW	25 h	Denegado	Denegado	PASA
Administrador	No	NEW	48 h	Permitido	Permitido	PASA
Propietario	Sí	NEW	24 h	Denegado	Denegado	PASA
Propietario	Sí	NEW	23 h 59 min 59 s	Permitido	Permitido	PASA

Rol	Dueño	Pedido	Antigüedad	Esperado	Real	Prueba
Tercero	No	CONFIRMED	1 h	Denegado	Denegado	PASA

Tabla 5. *Matriz de autorización (docs/matriz_autorizacion.csv).*

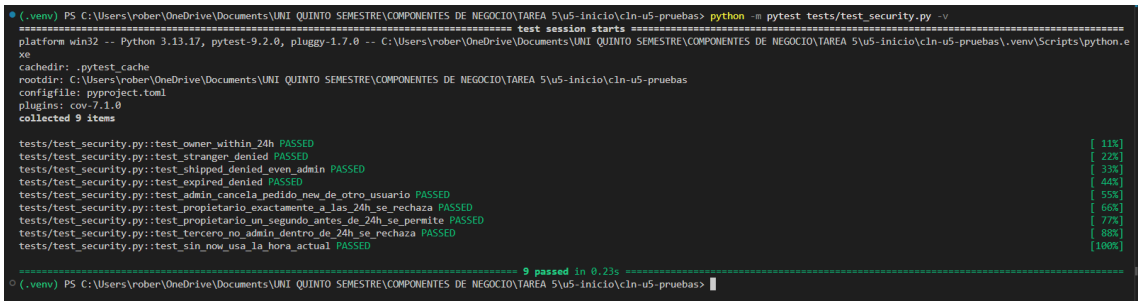


Figura 9. *Pruebas de autorización: 9 passed.*

Evidencia E6.

El caso de exactamente 24 horas es el límite clave: la condición es «menos de 24 horas», así que a las 24:00:00 se rechaza y a las 23:59:59 se permite. Probar los dos lados del límite demuestra que no se usó ≤ por error. Estas pruebas verifican **autorización** (quién puede hacer qué), no autenticación (quién es el usuario).

Casos del paquete leídos desde CSV

Como indica el paquete del laboratorio, los CSV no se cargan solos: se creó tests/test_casos_csv.py, que lee datos/casos_de_prueba.csv con csv.DictReader, lo une con datos/pedidos_prueba.csv y genera una prueba parametrizada por caso (CP01–CP12). CP13 a CP15 se cubren con las pruebas de la pasarela y del repositorio. El script generar_matrices.py ejecuta los 15 casos contra el código y escribe el resultado real en docs/matriz_pruebas.csv.

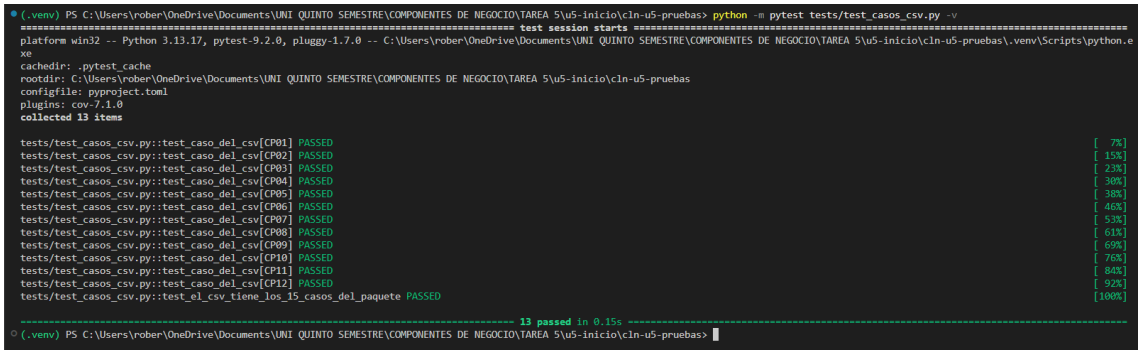


Figura 10. *Casos CP01–CP12 leídos del CSV: 13 passed.*

Matriz de pruebas: resultado esperado contra resultado real

Caso y tipo	Entrada	Resultado esperado	Resultado real	Estado	Prueba
CP01 Éxito	total=120, vip=0, deuda=0, estado=NEW	CONFIRMED, cobro 120.00	CONFIRMED, cobro 120.00	PASA	test_casos_csv
CP02 Éxito	total=200, vip=1, deuda=0, estado=NEW	CONFIRMED, cobro 170.00	CONFIRMED, cobro 170.00	PASA	test_casos_csv
CP03 Error	total=150, vip=0, deuda=500, estado=NEW	CustomerHasDebt	CustomerHasDebt: outstanding debt	PASA	test_casos_csv
CP04 Límite	total=0, vip=0, deuda=0, estado=NEW	ValueError	ValueError: total must be positive	PASA	test_casos_csv
CP05 Error	total=-10, vip=0, deuda=0, estado=NEW	ValueError	ValueError: total must be positive	PASA	test_casos_csv
CP06 Éxito	total=1000, vip=1, deuda=0, estado=NEW	CONFIRMED, cobro 850.00	CONFIRMED, cobro 850.00	PASA	test_casos_csv
CP07 Límite	total=0.01, vip=1, deuda=0, estado=NEW	CONFIRMED, cobro 0.01	CONFIRMED, cobro 0.01	PASA	test_casos_csv
CP08 Error	total=250, vip=0, deuda=0, estado=CONFIRMED	OrderAlreadyConfirmed (ValueError): already confirmed	OrderAlreadyConfirmed: already confirmed	PASA	test_casos_csv
CP09 Éxito	total=300, vip=1, deuda=0, estado=NEW	CONFIRMED, cobro 255.00	CONFIRMED, cobro 255.00	PASA	test_casos_csv

Caso y tipo	Entrada	Resultado esperado	Resultado real	Estado	Prueba
CP10 Éxito	total=75, vip=0, deuda=0, estado=NEW	CONFIRMED, cobro 75.00	CONFIRMED, cobro 75.00	PASA	test_casos_csv
CP11 Error	total=500, vip=1, deuda=250, estado=NEW	CustomerHasDebt	CustomerHasDebt: outstanding debt	PASA	test_casos_csv
CP12 Límite	total=99.99, vip=1, deuda=0, estado=NEW	CONFIRMED, cobro 84.99	CONFIRMED, cobro 84.99	PASA	test_casos_csv
CP13 Error	Order(100), mock charge -> False	RuntimeError: payment rejected	RuntimeError: payment rejected	PASA	test_service
CP14 Éxito	save(1, Order(120)) + commit	Registro visible después de commit	Registro visible después de commit	PASA	test_repository
CP15 Error	save(1, Order(120)) + rollback	Registro ausente después de rollback	Registro ausente después de rollback	PASA	test_repository

Tabla 6. *Matriz de pruebas CP01–CP15 (docs/matriz_pruebas.csv): 6 de éxito, 6 de error y 3 límite.*

Observaciones. CP04 (total 0), CP07 (0.01 VIP: el descuento se redondea a 0.00) y CP12 (99.99 VIP: 14.9985 se redondea a 15.00) son los valores límite. CP08 es el caso TDD: el CSV describe el comportamiento anterior (se confirmaba dos veces) y ahora el resultado real es OrderAlreadyConfirmed. El proyecto base usa float para los importes; en la adaptación a CleanPro se usó Decimal para evitar errores de redondeo.

Paso 7 · Cobertura y calidad (E7)

Terminal

```
python -m ruff check src tests
python -m pytest --cov=src --cov-report=term-missing --cov-report=html --cov-fail-under=80
start htmlcov\index.html
```

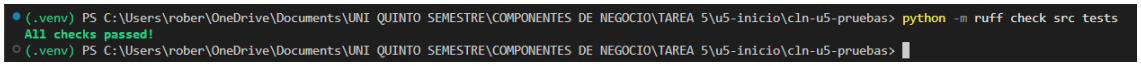


Figura 11. *Ruff sin errores.*

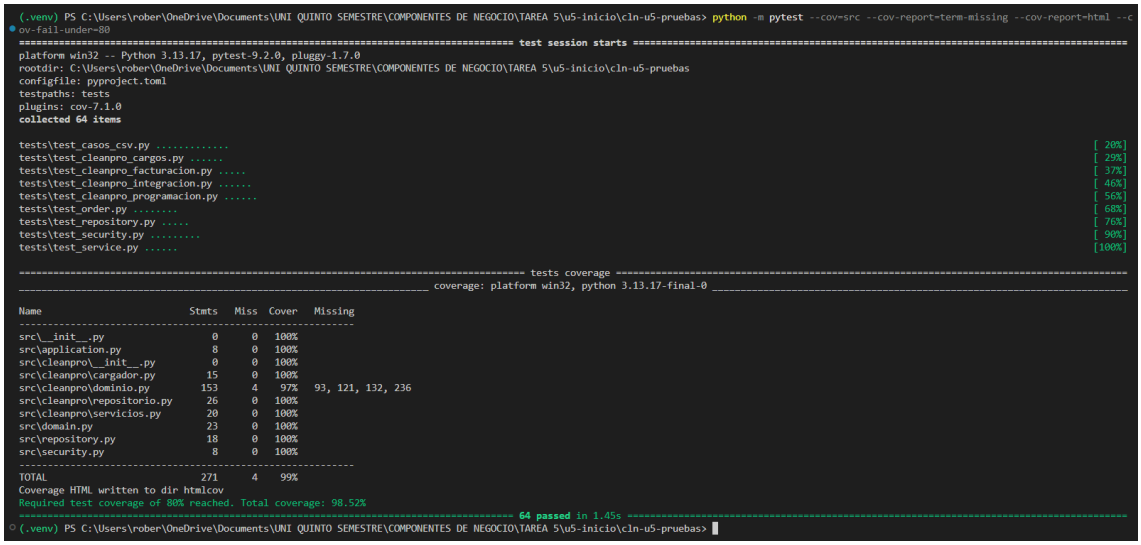


Figura 12. *Resumen de cobertura: 98.52 % (mínimo exigido 80 %) y 64 pruebas aprobadas.*

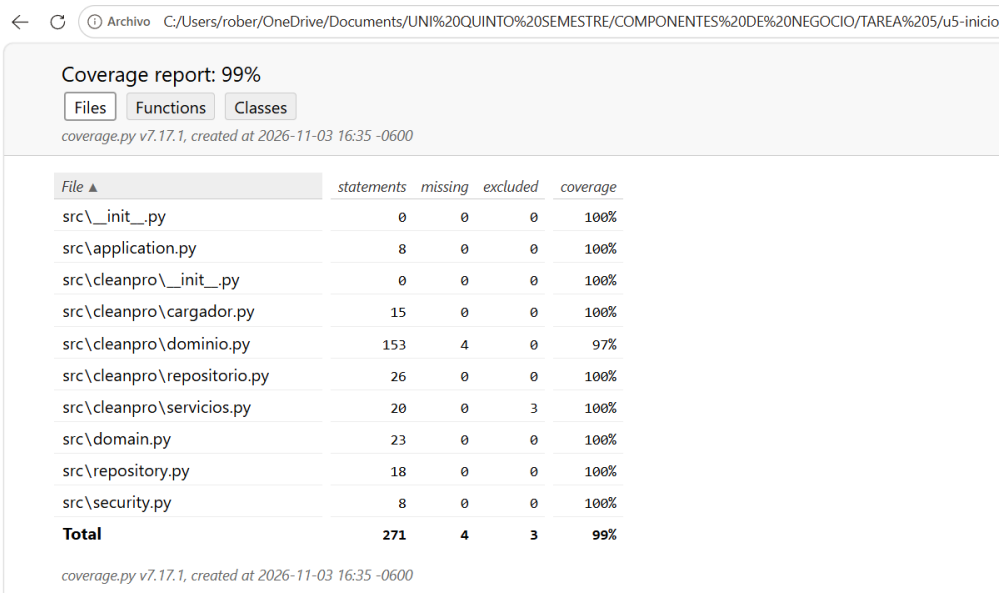


Figura 13. *Reporte HTML de cobertura (htmlcov/index.html).*

```
230     def autorizar_cargo(self, cargo: CargoAdicional, rol: Rol) -> None:
231         if rol != Rol.COORDINADOR:
232             raise AutorizacionDenegada(
233                 "RB-15", f"solo el coordinador autoriza cargos adicionales (rol: {rol.value})"
234             )
235         if cargo not in self.cargos:
236             raise ValueError("El cargo no pertenece a este servicio")
237         cargo.autorizado = True
238         self.eventos.append("CargoAdicionalAutorizado")
239
240     # ----- R2 + R3: facturar -----------------------------------------------
241     def calcular_factura(self) -> Decimal:
242         if self.estado == EstadoServicio.NO_REALIZADO:
243             return (self.total * PORCENTAJE_CANCELACION).quantize(CENTAVOS, ROUND_HALF_UP)
244         if self.estado != EstadoServicio.COMPLETADO:
```

Figura 14. *Línea 236 de src/cleanpro/dominio.py en rojo: rama sin probar.*

Evidencia E7.

La cobertura total es de 98.52 % sobre todo src/, sin excluir módulos, y alcanza el 100 % en src/domain.py y src/application.py, que el manual indica como alcance mínimo. El reporte marca cuatro líneas sin ejecutar, todas en src/cleanpro/dominio.py. La más representativa es la **línea 236**: autorizar un cargo que no pertenece al servicio lanza ValueError. Ninguna prueba intenta autorizar un cargo de otro servicio, así que esa defensa nunca se ejecutó. La prueba que faltaría crearía dos servicios y autorizaría en uno el cargo del otro, esperando el rechazo. Las otras líneas son la validación de cantidad ≤ 0 de una línea de servicio, el método abstracto de la Specification base y la evaluación booleana del AND, que se usa mediante incumplidas(). Se dejaron visibles a propósito: subir la cobertura con pruebas sin aserciones útiles no demostraría nada. Cobertura del 80 % significa que se ejecutó el 80 % de las líneas, no que se encontró el 80 % de los defectos.

Paso 8 · Automatizar la revisión con GitHub Actions (E8)

Repositorio: https://github.com/RobertocSanchezf23/cln_u5_pruebas_equipo-12

El archivo .github/workflows/ci.yml se ejecuta en cada push o pull request: instala Python 3.11 y las dependencias, corre Ruff, ejecuta pytest con umbral de cobertura del 80 %, compila src/ y publica el reporte HTML de cobertura como artefacto. El proyecto se subió en commits separados y el archivo del pipeline se agregó al final, para que la primera ejecución corriera sobre el proyecto completo.

#	Commit	Pipeline
1	Agrega proyecto base U5 con ajustes de configuración	—
2	Agrega .gitignore	—
3	Agrega pruebas unitarias, de integración, mocks, seguridad y CleanPro	—
4	Agrega datos del laboratorio y del PIA, matrices CSV y README	—
5	Agrega pipeline de GitHub Actions	Ejecución #1: verde
6	Prueba de bloqueo: aserción falsa intencional	Ejecución #2: rojo
7	Revierte la aserción falsa	Ejecución #3: verde

Tabla 7. *Historial de commits del repositorio.*

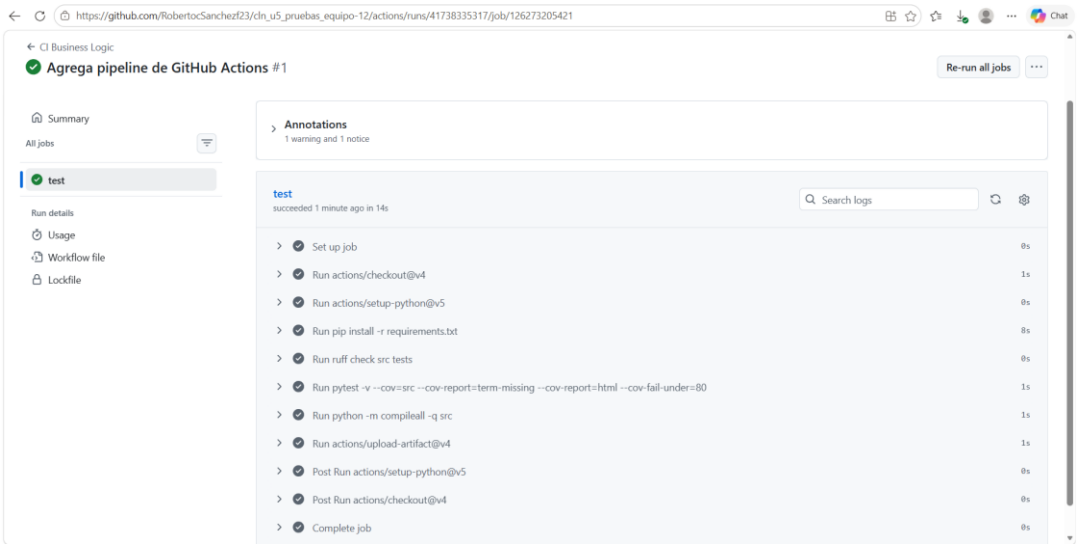


Figura 15. *Ejecución #1 en verde: todos los pasos del pipeline aprobados.*

Para comprobar que el pipeline bloquea cambios defectuosos, se cambió en GitHub la aserción de test_vip_discount de 85 a 90, un valor falso a propósito:

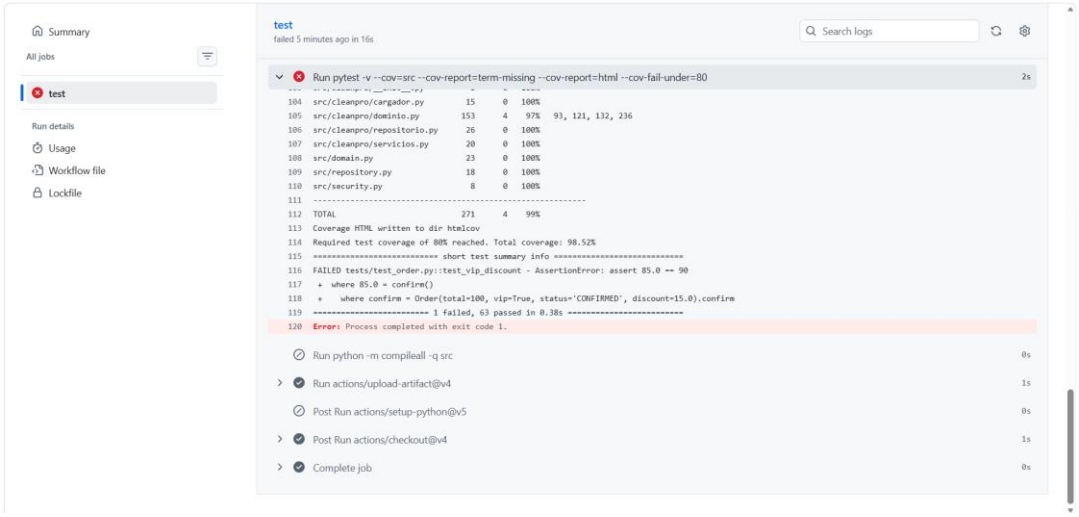


Figura 16. *Ejecución #2 en rojo: pytest detecta AssertionError: assert 85.0 == 90 (1 failed, 63 passed) y el job termina con código 1.*

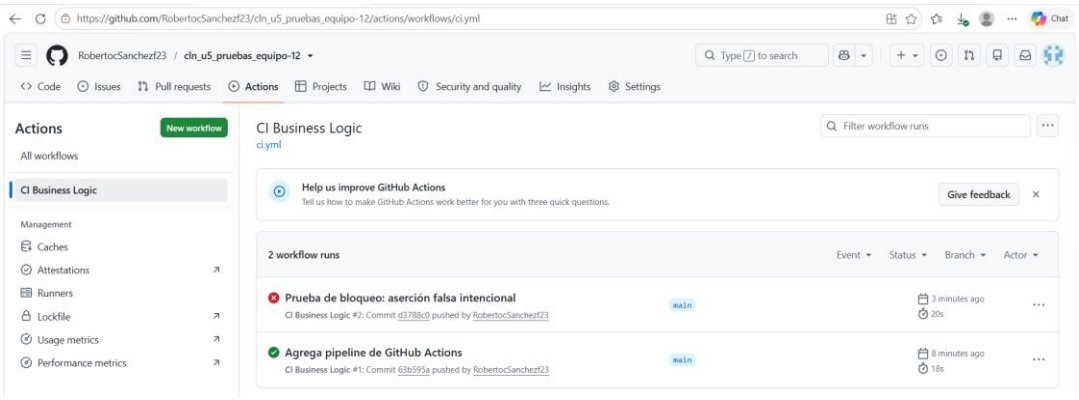


Figura 17. *Lista de ejecuciones: #1 verde y #2 roja.*

Después se regresó la aserción a 85 y el pipeline volvió a verde:

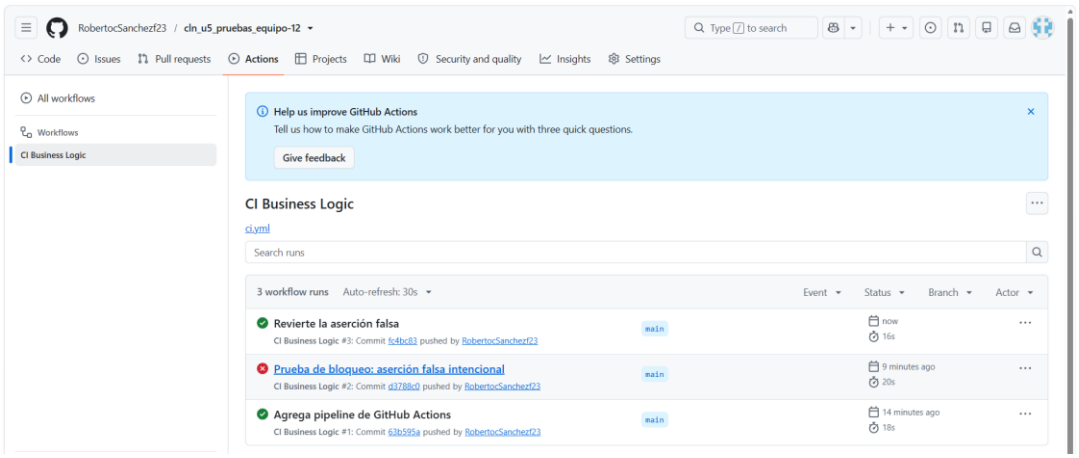


Figura 18. *Lista de ejecuciones: verde, roja y nuevamente verde.*

Evidencia E8.

En la ejecución #2 el pipeline detectó que el cobro real de un pedido VIP de $100 es 85.0 y no 90: la prueba test_vip_discount falló, pytest terminó con código 1 y el job se marcó como fallido. Aunque la cobertura seguía en 98.52 %, el paso falló por la aserción. Una cobertura alta no evita que una regla incorrecta llegue al repositorio, pero una prueba con una aserción precisa sí. Para que GitHub impida fusionar cambios en rojo se necesita configurar branch protection o rulesets en la rama main; tener el archivo YAML no activa ese bloqueo por sí solo.

Paso 9 · Aplicación a la empresa del PIA: CleanPro (E9)

CleanPro presta servicios de limpieza residencial y empresarial. Se eligieron tres reglas del **BRD del equipo** (Unidad 2) que están relacionadas con los problemas principales de la empresa: los errores de facturación (PA-02) y los servicios adicionales que no llegan a administración (PA-04). Las reglas se implementaron en Python en src/cleanpro, con la misma estructura de la Unidad 3: un Aggregate Servicio, Value Objects, Specifications, un repositorio SQLAlchemy y un Application Service con un puerto de notificación. Los importes usan Decimal. Los datos de prueba son los del paquete del PIA entregado por el docente: 40 contratantes, 20 servicios del catálogo y 60 solicitudes.

Regla	BRD	Implementación	Casos
R1. Solo se programa un servicio cuya cotización aceptó el cliente; además, el cliente debe estar activo y la solicitud debe tener servicios habilitados	RB-02 + precondiciones del paquete	Servicio.programar() con 4 Specifications combinadas con AND	CLN-CP01 a CP05
R2. Solo se factura un servicio terminado; si el cliente no permite el acceso tras 15 minutos, el servicio queda no realizado y se cobra el 50 %	RB-07, RB-11	calcular_factura() y registrar_acceso_denegado()	CLN-CP06 a CP09
R3. Un cargo adicional debe autorizarse antes de facturar, y solo el coordinador puede autorizarlo	RB-12, RB-14, RB-15	registrar_cargo(), autorizar_cargo(rol) y calcular_factura()	CLN-CP10 a CP13
Persistencia y aviso al cliente	US-03, US-08	SqlServicioRepository y ProgramacionService con notificador simulado	CLN-CP14 a CP18

Tabla 8. *Reglas de CleanPro adaptadas y probadas.*

src/cleanpro/dominio.py

```
PUEDE_PROGRAMARSE = (
    CotizacionAceptada() & ClienteActivo()
    & TieneServicios() & ServiciosHabilitados()
)

def calcular_factura(self) -> Decimal:
    if self.estado == EstadoServicio.NO_REALIZADO:  # RB-11: 50 %
        return (self.total * PORCENTAJE_CANCELACION).quantize(
            CENTAVOS, ROUND_HALF_UP)
    if self.estado != EstadoServicio.COMPLETADO:    # RB-07
        raise FacturacionNoPermitida(
            "RB-07", "solo se factura un servicio terminado")
    if any(not c.autorizado for c in self.cargos):  # RB-14
        raise FacturacionNoPermitida(
            "RB-14", "hay cargos adicionales sin autorizar")
    cargos = sum((c.monto for c in self.cargos), Decimal("0.00"))
    return (self.total + cargos).quantize(CENTAVOS, ROUND_HALF_UP)
```

```
(.venv) PS C:\Users\rober\OneDrive\Documents\UNI QUINTO SEMESTRE\COMPONENTES DE NEGOCIO\TAREA 5\u5-inicio\cln-u5-pruebas> python -m pytest -k cleanpro -v
============================= test session starts ==============================
platform win32 -- Python 3.13.17, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\rober\OneDrive\Documents\UNI QUINTO SEMESTRE\COMPONENTES DE NEGOCIO\TAREA 5\u5-inicio\cln-u5-pruebas\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\rober\OneDrive\Documents\UNI QUINTO SEMESTRE\COMPONENTES DE NEGOCIO\TAREA 5\u5-inicio\cln-u5-pruebas
configfile: pyproject.toml
testpaths: tests
plugins: cov-7.1.0
collected 64 items / 41 deselected / 23 selected

tests/test_cleanpro_cargos.py::test_cln_cp10_exito_cargo_autorizado_se_suma_a_la_factura PASSED [  4%]
tests/test_cleanpro_cargos.py::test_cln_cp11_error_cargo_sin_autorizar_bloquea_la_factura PASSED [  8%]
tests/test_cleanpro_cargos.py::test_cln_cp12_error_solo_el_coordinador_autoriza[SUPERVISOR] PASSED [ 13%]
tests/test_cleanpro_cargos.py::test_cln_cp12_error_solo_el_coordinador_autoriza[ADMINISTRACION] PASSED [ 17%]
tests/test_cleanpro_cargos.py::test_cln_cp13_limite_un_centavo_sin_autorizar_tambien_bloquea PASSED [ 21%]
tests/test_cleanpro_cargos.py::test_cln_cargo_en_cero_no_es_valido PASSED [ 26%]
tests/test_cleanpro_facturacion.py::test_cln_cp06_exito_servicio_completado_se_factura_completo PASSED [ 30%]
tests/test_cleanpro_facturacion.py::test_cln_cp07_error_servicio_programado_no_se_factura PASSED [ 34%]
tests/test_cleanpro_facturacion.py::test_cln_cp08_limite_15_minutos_no_realizado_cobra_50 PASSED [ 39%]
tests/test_cleanpro_facturacion.py::test_cln_cp09_limite_14_minutos_aun_no_es_no_realizado PASSED [ 43%]
tests/test_cleanpro_facturacion.py::test_cln_no_se_completa_sin_iniciar PASSED [ 47%]
tests/test_cleanpro_integracion.py::test_cln_cp14_commit_el_servicio_programado_queda_guardado PASSED [ 52%]
tests/test_cleanpro_integracion.py::test_cln_cp15_rollback_el_servicio_no_queda_guardado PASSED [ 56%]
tests/test_cleanpro_integracion.py::test_cln_cp16_mock_notifica_al_cliente_una_vez PASSED [ 60%]
tests/test_cleanpro_integracion.py::test_cln_cp17_si_la_regla_falla_no_se_guarda_ni_se_notifica PASSED [ 65%]
tests/test_cleanpro_integracion.py::test_cln_cp18_si_falla_el_aviso_el_servicio_sigue_programado PASSED [ 69%]
tests/test_cleanpro_integracion.py::test_el_paquete_tiene_60_solicitudes_en_solicitado PASSED [ 73%]
tests/test_cleanpro_programacion.py::test_cln_cp01_exito_servicio_aceptado_se_programa PASSED [ 78%]
tests/test_cleanpro_programacion.py::test_cln_rechazos_al_programar[CP02-no-aceptada] PASSED [ 82%]
tests/test_cleanpro_programacion.py::test_cln_rechazos_al_programar[CP03-cliente-inactivo] PASSED [ 86%]
tests/test_cleanpro_programacion.py::test_cln_rechazos_al_programar[CP04-sin-detalle] PASSED [ 91%]
tests/test_cleanpro_programacion.py::test_cln_rechazos_al_programar[CP05-deshabilitado] PASSED [ 95%]
tests/test_cleanpro_programacion.py::test_cln_no_se_programa_dos_veces PASSED [100%]

======================= 23 passed, 41 deselected in 0.44s ======================
(.venv) PS C:\Users\rober\OneDrive\Documents\UNI QUINTO SEMESTRE\COMPONENTES DE NEGOCIO\TAREA 5\u5-inicio\cln-u5-pruebas>
```

Figura 19. *Pruebas de CleanPro: 23 passed.*

```
(.venv) PS C:\Users\rober\OneDrive\Documents\UNI QUINTO SEMESTRE\COMPONENTES DE NEGOCIO\TAREA 5\u5-inicio\cln-u5-pruebas> python generar_matrices.py
matriz_pruebas.csv: 15/15 casos PASA
matriz_autorizacion.csv: 8/8 casos PASA
matriz_cleanpro.csv: 18/18 casos PASA
(.venv) PS C:\Users\rober\OneDrive\Documents\UNI QUINTO SEMESTRE\COMPONENTES DE NEGOCIO\TAREA 5\u5-inicio\cln-u5-pruebas>
```

Figura 20. *Generación de las matrices: todos los casos en PASA.*

Matriz de casos de CleanPro

Caso	Regla	Entrada	Resultado esperado	Resultado real	Estado
CLN-CP01 CLN-O001 Éxito	R1 RB-02	Cliente activo, cotización aceptada, 2 servicios	PROGRAMADO	PROGRAMADO	PASA
CLN-CP02 CLN-O008 Error	R1 RB-02	Cotización no aceptada (autorizado=0)	ServicioNoProgramable	ServicioNoProgramable (RB-02): el cliente no ha aceptado la cotización	PASA
CLN-CP03 CLN-O022 Error	R1 precondición	Cliente C007 inactivo	ServicioNoProgramable	ServicioNoProgramable (PIA-precondición): el cliente está inactivo	PASA
CLN-CP04 CLN-O013 Límite	R1 precondición	Solicitud con 0 servicios	ServicioNoProgramable	ServicioNoProgramable (PIA-precondición): la solicitud no tiene servicios en el detalle	PASA
CLN-CP05 CLN-O005 Error	R1 precondición	Incluye P011 deshabilitado	ServicioNoProgramable	ServicioNoProgramable (PIA-precondición): incluye un servicio deshabilitado en el catálogo	PASA
CLN-CP06 CLN-O006 Éxito	R2 RB-07	Programar, iniciar y completar	Factura 2840.00	Factura 2840.00	PASA
CLN-CP07 CLN-O007 Error	R2 RB-07	Facturar estando PROGRAMADO	FacturacionNoPermitida	FacturacionNoPermitida (RB-07): solo se factura un servicio terminado (estado: PROGRAMADO)	PASA

Caso	Regla	Entrada	Resultado esperado	Resultado real	Estado
CLN-CP08 CLN-O003 Límite	R2 RB-11	Acceso denegado tras 15 min (1,491.00)	Factura 745.50	Factura 745.50	PASA
CLN-CP09 CLN-O004 Límite	R2 RB-11	Acceso denegado tras 14 min	TransicionInvalida	TransicionInvalida (RB-11): se debe esperar al menos 15 minutos (se esperaron 14)	PASA
CLN-CP10 CLN-O010 Éxito	R3 RB-14	Cargo 350.00 autorizado por coordinador	Factura 7095.00	Factura 7095.00	PASA
CLN-CP11 CLN-O011 Error	R3 RB-14	Cargo 500.00 sin autorizar	FacturacionNoPermitida	FacturacionNoPermitida (RB-14): hay cargos adicionales sin autorizar	PASA
CLN-CP12 CLN-O012 Error	R3 RB-15	Supervisor intenta autorizar cargo	AutorizacionDenegada	AutorizacionDenegada (RB-15): solo el coordinador autoriza cargos adicionales (rol: SUPERVISOR)	PASA
CLN-CP13 CLN-O014 Límite	R3 RB-14	Cargo de 0.01 sin autorizar	FacturacionNoPermitida	FacturacionNoPermitida (RB-14): hay cargos adicionales sin autorizar	PASA
CLN-CP14 CLN-O001 Integración	Persistencia	Programar y hacer commit	PROGRAMADO guardado	PROGRAMADO guardado; avisos=1; pendientes=0	PASA

Caso	Regla	Entrada	Resultado esperado	Resultado real	Estado
CLN-CP15 CLN-O002 Integración	Persistencia	Guardar y hacer rollback	no guardado	no guardado tras rollback	PASA
CLN-CP16 CLN-O001 Integración	Mock notificador	Aviso al cliente al programar	avisos=1	PROGRAMADO guardado; avisos=1; pendientes=0	PASA
CLN-CP17 CLN-O008 Integración	Mock notificador	Regla falla: no guarda ni avisa	ServicioNoProgramable; guardado=False	ServicioNoProgramable; guardado=False	PASA
CLN-CP18 CLN-O006 Integración	Mock notificador	Falla el aviso (timeout)	pendientes=1	PROGRAMADO guardado; avisos=1; pendientes=1	PASA

Tabla 9. *Matriz de CleanPro (docs/matriz_cleanpro.csv): 18 casos, por regla de éxito, error y límite, más integración.*

Evidencia E9 · Decisiones de diseño.

- **Reglas del BRD, no del paquete.** El paquete del PIA propone su propio modelo y pide no reemplazar los requerimientos del equipo. Por eso se usaron RB-02, RB-07, RB-11, RB-12, RB-14 y RB-15, y las diferencias se documentaron en la tabla siguiente.
- **Valores límite reales.** RB-11 dice «esperar hasta 15 minutos»: con 15 minutos el servicio queda no realizado (CP08) y con 14 se rechaza (CP09). Para RB-14 se probó que un cargo de solo $0.01 sin autorizar también bloquea la factura (CP13): la regla depende de la autorización, no del monto.
- **Persistencia.** Los totales se guardan en centavos (entero) para no perder exactitud en SQLite. CP14 verifica el commit y CP15 el rollback, cada uno con su propia base en memoria.
- **Mock del notificador.** US-03 pide avisar al cliente al programar. El Application Service primero valida, guarda y confirma la transacción, y después notifica. Si la regla falla, no se guarda ni se avisa (CP17). Si el aviso falla, la programación sigue siendo válida y queda pendiente de reintento (CP18). Así se evita el riesgo de consistencia encontrado en el Paso 5.
- **Cobertura del código pertinente.** src/cleanpro alcanza 97 % en el dominio y 100 % en el cargador, el repositorio y el servicio.

Modelo propuesto en el paquete del PIA	Lo que se implementó según el BRD de CleanPro
autorizado	El cliente aceptó la cotización (RB-02)
cliente_activo y detalle no vacío	No están en el BRD; se adoptaron como precondiciones de programación
Recurso con habilitado = 0	No está en el BRD; se adoptó como precondición: un servicio deshabilitado del catálogo predefinido no se programa
Estados SOLICITADO → PROGRAMADO → EN_PROCESO → COMPLETADO	Se conservaron; COMPLETADO equivale a «servicio terminado» (RB-05, RB-07)
CANCELADO desde SOLICITADO	No se incluyó en esta muestra; el BRD contempla NO_REALIZADO con cargo del 50 % (RB-11)
beneficio_referencia: preferente, total ≥ 1500 y sin promoción	No existe en el BRD y no se implementó; el BRD solo contempla 10 % por frecuencia recurrente (RB-03)

Tabla 10. *Diferencias entre el modelo propuesto del paquete y el BRD del equipo.*

Uso de inteligencia artificial

Se usó IA para proponer casos de éxito, error y límite, fixtures y mocks, con un prompt basado en el sugerido por la guía: «Analiza esta clase de dominio y estas reglas aprobadas. Propón casos de prueba pytest para éxito, error y límites. No inventes reglas nuevas. Señala supuestos, riesgos y ramas sin cobertura». Cada propuesta se verificó contra las reglas y se ejecutó antes de aceptarla.

Propuesta de la IA	Decisión del equipo	Motivo
Implementar en CleanPro la regla «beneficio_referencia» del paquete (preferente, ≥ 1500, sin promoción)	Rechazada	No está en el BRD de CleanPro y el paquete pide no sustituir los requerimientos propios
Relacionar el servicio deshabilitado con SUP-06 como «fuera del catálogo vigente»	Corregida	SUP-06 habla del catálogo de extras; se documentó como precondición adoptada del paquete
Comparar el resultado de CP08 contra «ValueError» en el generador de matrices	Corregida	Después del refactor la excepción real es OrderAlreadyConfirmed (subclase de ValueError); la primera versión marcaba el caso como FALLA
Pruebas de límite a 24 h, rollback parcial, unicidad, timeout y riesgo de pago	Aceptadas	Corresponden a reglas implementadas y se verificaron ejecutándolas

Tabla 11. *Casos sugeridos por IA que el equipo aceptó, corrigió o rechazó.*

Autoevaluación

Pregunta de la guía	Respuesta
¿Puedo explicar por qué una prueba unitaria no requiere base de datos?	Sí: el dominio no importa SQLAlchemy; la regla se evalúa sola (Paso 2)
¿Tengo evidencia real de RED y GREEN?	Sí: Figuras 3, 4 y 5
¿Sé diferenciar commit y rollback?	Sí: Paso 4, incluido el rollback por unicidad
¿Demostré que el mock no genera un cargo real?	Sí: test_mock_no_hace_cargos_reales
¿Comprobé permisos y límites temporales?	Sí: matriz de autorización, 24 h exactas y 23:59:59
¿Mi cobertura alcanza 80 % sin ocultar módulos?	Sí: 98.52 % sobre todo src/
¿Mi pipeline se ejecutó en GitHub y detectó un fallo?	Sí: verde, rojo y verde (Paso 8)

Pregunta de la guía	Respuesta
¿Mis nueve casos del PIA representan reglas reales de mi empresa?	Sí: 13 casos de reglas del BRD (RB-02, RB-07, RB-11, RB-12, RB-14, RB-15) y 5 de integración

Tabla 12. *Autoevaluación antes de entregar.*

Reflexión

El defecto más costoso que podría llegar a producción sería cobrar mal, sobre todo cobrar o confirmar algo que no debía cobrarse. En el proyecto base, la prueba de caracterización demostró que un pedido queda confirmado aunque la pasarela rechace el pago. En producción eso significaría enviar mercancía sin haberla cobrado, y el error solo aparecería semanas después, al conciliar las cuentas, cuando ya hay pérdidas y clientes inconformes. En CleanPro el riesgo equivalente es facturar un servicio con cargos adicionales que el coordinador nunca autorizó, o cobrar el 100 % de un servicio que no se realizó. Es exactamente el problema de facturación que describe el BRD. Las pruebas que lo evitan son las que verifican la regla en el dominio y no en la pantalla: test_cln_cp11 y test_cln_cp13 bloquean la factura con cualquier cargo pendiente, aunque sea de un centavo, y test_cln_cp08 garantiza el 50 %. Como se ejecutan en cada push, el pipeline rechazaría cualquier cambio que rompiera esas reglas antes de que llegara a un cliente.

Conclusiones:

La actividad mostró la diferencia entre un programa que se ejecuta y uno que tiene evidencia de que funciona. Las 64 pruebas cubren las reglas del dominio, la persistencia con commit y rollback, la pasarela de pago simulada, la política de cancelación y los casos del paquete leídos desde CSV. Las matrices dejan registro del resultado esperado y del resultado real de cada caso. El ciclo TDD agregó una regla nueva con evidencia de su fase roja, y el pipeline de GitHub Actions demostró que detecta y frena un cambio defectuoso.

Las pruebas también encontraron problemas que no se veían a simple vista: el pipeline original habría fallado por la configuración del path y por Ruff, y el servicio de pago puede dejar un pedido confirmado sin cobro. Al aplicar lo aprendido a CleanPro, las reglas de facturación del BRD quedaron protegidas con casos de éxito, error y límite usando los datos del PIA. Esa base de código y de pruebas se reutilizará en el proyecto integrador.

Conclusiones individuales:

Roberto Carlos Sánchez Flores

Lo que más me sirvió fue hacer TDD de verdad. Ver la prueba en rojo antes de escribir la solución me hizo entender que la prueba es la que define el comportamiento, y no algo

que se agrega al final para cumplir. También aprendí a probar los valores límite con cuidado: el caso del propietario a exactamente 24 horas y el de 23:59:59 me mostraron que un solo signo, < o ≤, cambia una regla de negocio. Con fechas fijas las pruebas siempre dan el mismo resultado.

Donovan Adrian Serrato Lores

Para mí lo más importante fue entender las pruebas de integración y los mocks. Con SQLite en memoria comprobé que el commit guarda y el rollback deshace sin tocar ninguna base real, y con el mock de la pasarela pude simular pagos aprobados, rechazados y hasta un timeout sin mover dinero. Gracias a eso encontramos que el pedido se confirma aunque el pago falle. También me quedó claro para qué sirve GitHub Actions: cuando cambiamos la aserción a propósito, el pipeline se puso en rojo solo y nos dijo exactamente qué prueba falló.

Laura Sofía Vargas Ramírez

Me tocó entender cómo pasar lo aprendido a CleanPro. Elegir reglas reales de nuestro BRD, y no las del paquete, me obligó a revisar qué decía exactamente cada una, por ejemplo que el cargo de cancelación del 50 % solo aplica después de 15 minutos de espera. También aprendí que la cobertura no lo es todo: con 98 % todavía hay ramas sin probar, y el pipeline detectó la aserción falsa aunque la cobertura no cambió. Revisar con cuidado lo que propone la IA fue necesario, porque algunas sugerencias no coincidían con nuestro BRD.

Repositorio de GitHub:

El repositorio contiene src/ (proyecto base y adaptación a CleanPro), tests/, datos/ (CSV del laboratorio y del PIA), docs/ (matrices CSV), generar_matrices.py, el pipeline en .github/workflows/ci.yml y un README con instalación, comandos y arquitectura.

https://github.com/RobertocSanchezf23/cln_u5_pruebas_equipo-12